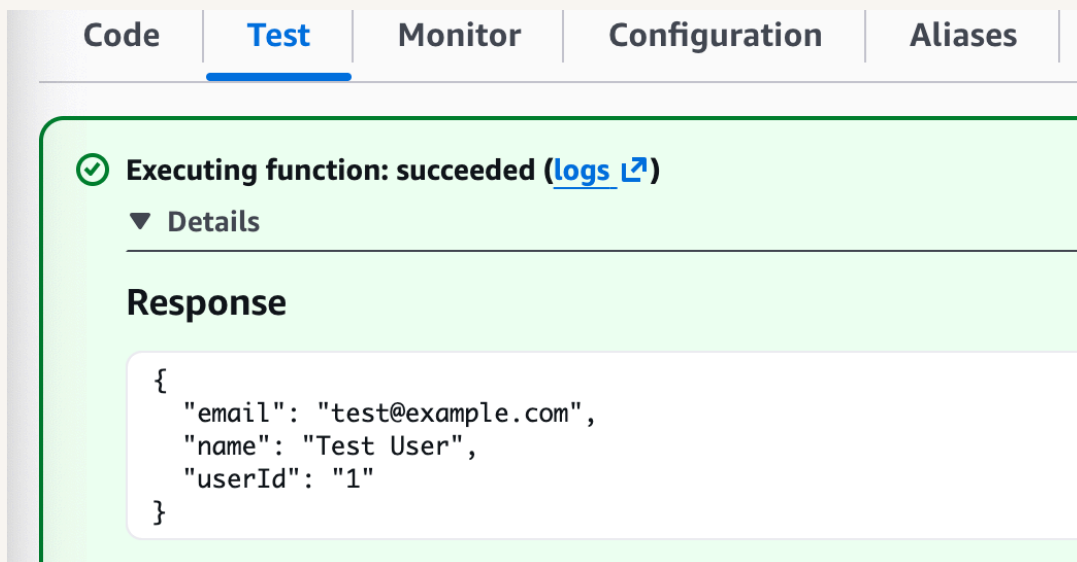




Fetch Data with AWS Lambda



Seye Sanya





Introducing Today's Project!

In this project, I will demonstrate how to use AWS Lambda to retrieve data that is in a DynamoDB database. I'm doing this project to learn how to set up the data tier of a web app and connect that with a Lambda function as well.

Tools and concepts

Services I used were Amazon DynamoDB and AWS Lambda. Key concepts I learnt include Lambda functions, adding table items, testing Lambda functions, how to select the right permission policy, writing custom inline policies.

Project reflection

This project took me approximately two hours including demo time. The most challenging part was writing my own inline policy. It was most rewarding to see the right data pop when I ran a successful function test.

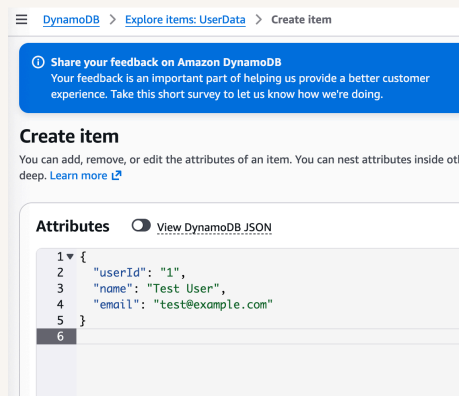
I chose to do this project today because I wanted to learn about the data tier of a three-tier architecture. This project surely met my goals as I learnt how to connect it to AWS Lambda.



Project Setup

To set up my project, I created a database using DynamoDB. DynamoDB is a NoSQL database, so it is very fast at retrieval and flexible for data storage. The partition key is `userId`, which means the key identified for each data is its `userId`.

In my DynamoDB table, I added a piece of data. DynamoDB is schemaless, which means that the data we add can come with new attributes, and every piece of data can have its set of attributes without affecting other data.



AWS Lambda

AWS Lambda is a service that lets us run code without having to use servers. I'm using Lambda in this project to create a function, like a piece of code, that retrieves data from a DynamoDB table.



AWS Lambda Function

My Lambda function has an execution role, which is the set of permissions that it has to interact with other AWS services. By default, the role grants basic Lambda permissions, which are the permissions to write logs with CloudWatch, for example, success and error logs.

My Lambda function will retrieve data from a DynmoDB database table. The first half of my code is all about the data retrieval. The second half of the code is all about sending responses back e.g success messages, item not found, or general error messages.

The code uses AWS SDK, which is a set of tools that makes it easier for developers to use AWS in thier application code. My code uses SDK to get access to DynamoDB actions, like read or retrieve data items.

```
RetrieveUserData1

JS index.mjs x
JS index.mjs > ...
1 import { DynamoDBClient } from "@aws-sdk/client-dynamodb";
2 import { DynamoDBDocumentClient, GetCommand } from "@aws-sdk/lib-dynamodb";
3
4 const ddbClient = new DynamoDBClient({ region: 'us-east-1' }); // Make sure to replace this with your actual AWS region
5 const ddb = DynamoDBDocumentClient.from(ddbClient);
6
7 async function handler(event) {
8   const userId = String(event.userId); // Make sure to extract userId from the event
9   const params = {
10     TableName: 'UserData',
11     Key: { userId }
12   };
13
14   try {
15     const command = new GetCommand(params);
16     const data = await ddb.send(command); // Log the raw response from DynamoDB
17     console.log("DynamoDB Response:", JSON.stringify(data));
18     const { Item } = data;
19     if (Item) {
20       console.log("User data retrieved:", Item);
21       return Item;
22     } else {
23       console.log("No user data found for userId:", userId);
24       return null;
25     }
26   } catch (err) {
27     console.error("Unable to retrieve data:", err);
28     return err;
29   }
}
```

Function Testing

To test whether my Lambda function works, I wrote a function test in the Test tab in the Lambda console. The test is written in JSON. If the test is successful, I'd see the correct piece of data in JSON when we query for userId=1.

The test displayed a 'success' because the function itself ran with no errors, meaning there were no syntax or dependency issues but the function's response was actually an error because my Lambda function is retrieving data from DynamoDB without permissions.

Lambda > Functions > RetrieveUserData1

Successfully updated the function "RetrieveUserData1".

CodeTestMonitorConfigurationAliasesVersions

Executing function: succeeded (logs)

Details

Response

Format JSON

```
{  "$fault": "client",  "$metadata": {    "statusCode": 400,    "requestId": "RR4GAK0MC152IM918JMQN960NJVV4KQNS05AEMVJF66Q9ASUAAJG",    "attempts": 1,    "totalRetryDelay": 0  },  "name": "AccessDeniedException",  "__type": "com.amazon.coral.service#AccessDeniedException",  "message": "User: arn:aws:sts::201701436671:assumed-role/RetrieveUserData1-role-wuboliif/RetrieveUserData1 is not authorized to perform: dynamodb:GetItem on resource: arn:aws:dynamodb:us-east-1:201701436671:table/UserData because no identity-based policy allows the dynamodb:GetItem action"}
```



Function Permissions

To resolve the AccessDenied error, we are providing my Lambda function with the permissions to perform `*GetItem*` on a DynamoDB table because `GetItem` is the action that Lambda is currently blocked from doing.

There were four DynamoDB permission policies I could choose from, but I didn't pick `AWSLambdaDynamoDBExecutionRole` or `AWSLambdaInvocationDynamoDB` because they relate to DynamoDB streams, which speaks to live updates of table items that have changed.

I also didn't pick `AmazonDynamoDBFullAccess` because it would give my `:lambda` function the permission to do many things with the table.

`AmazonDynamoDBReadOnlyAccess` was the right choice because it secures my table from unauthorised access through Lambda.

Attach policy to RetrieveUserData1-role-wuboliif

▼ Current permissions policies (2)

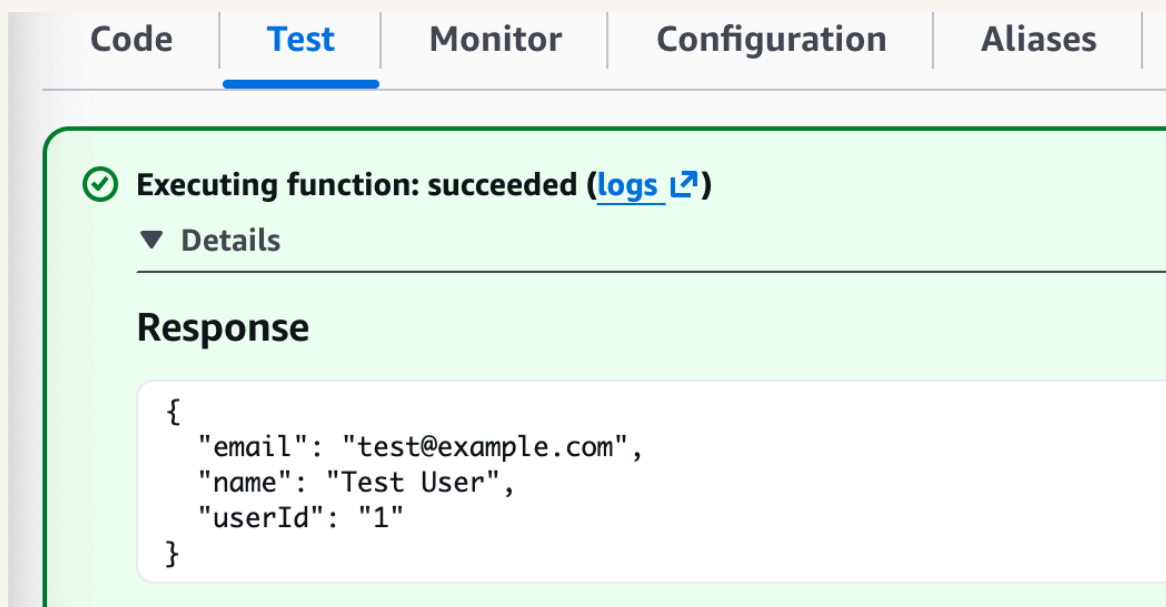
Policy name	▲	Type	▼
  AmazonDynamoDBReadOnlyAccess		AWS managed	



Final Testing and Reflection

To validate my new permission settings, I re-tested my Lambda function. The results were a piece of data and its three attributes being email, name and userId because that is the exact data that we inputted in our database at the start of the project.

Web apps are a popular use case of using Lambda and DynamoDB. For example, I could use Lambda to retrieve product information, user profiles, content like blog or news articles that are based on user queries or actions in my web app.





Enhancing Security

For my project extension, I challenged myself to replace the permission policy I granted my Lambda function previously. Instead of the AWS managed policy, I'll make sure it can only access UserData. This will enhance DynamoDB's security.

To create the permission policy, I used the visual editor because this was a more user-friendly and guided option for creating my custom policy. This involved checkboxes and dropdowns that I picked, and the policy was written in the background.

When updating a Lambda function's permission policies, you could risk losing access to resources or actions that it needs to work. I validated that my Lambda function still works by re-testing it after updating its permission policies, and yes, it still works.



▼ DynamoDB

Allow 1 Action



Specify what actions can be performed on specific resources in DynamoDB.

▼ Actions allowed

Specify actions from the service to be allowed.

Effect

☒ Allow ☐ Deny

Manual actions | [Add actions](#)

☐ All DynamoDB actions (dynamodb:*)

Access level

[Expand all](#) | [Collapse all](#)

▶ List (6)

▶ Read (**Selected 1/30**)

▶ Write (38)

▶ Permissions management (3)

▶ Tagging (2)



nextwork.ai

The place to learn & showcase your skills

Check out nextwork.ai for more projects

